Progetto di Esame — Elementi di Intelligenza Artificiale

Ottimizzazione Automatica dell'Architettura di Reti Neurali tramite Algoritmo Genetico

Corso di Laurea Triennale in Ingegneria Informatica

3 aprile 2026

Sommario

Questo documento descrive il progetto realizzato per l'esame di Elementi di Intelligenza Artificiale. Il sistema implementato combina una rete neurale costruita da zero in Python con un Algoritmo Genetico (GA) per automatizzare la ricerca della migliore architettura della rete sul dataset Iris. L'approccio rientra nel campo della Neural Architecture Search (NAS) e dimostra come due paradigmi di ottimizzazione — il calcolo del gradiente e l'evoluzione biologica — possano collaborare per risolvere un problema di classificazione multi-classe.

Indice

1 Introduzione **4**
1.1 Originalità e Rilevanza Pratica 4

2 Background Teorico **5**
2.1 Reti Neurali Artificiali 5
2.1.1 Struttura di un Neurone 5
2.1.2 Funzioni di Attivazione 5
2.1.3 Backpropagation 7
2.2 Algoritmi Genetici 8
2.2.1 Concetti Fondamentali 8
2.2.2 Il Ciclo Evolutivo 8
2.2.3 Selezione Tramite Torneo 9
2.3 Neural Architecture Search 9

3 Implementazione **11**
3.1 Struttura del Progetto 11
3.2 Rete Neurale (`nn_engine.py`) 11
3.2.1 Modifica al Costruttore 11
3.2.2 Funzioni di Attivazione per Layer 11
3.3 Algoritmo Genetico (`genetic_algorithm.py`) 12
3.3.1 Rappresentazione del Cromosoma 12
3.3.2 Funzione di Fitness con Penalità di Complessità 12
3.3.3 Crossover 13
3.3.4 Mutazione 14
3.3.5 Elitismo nel metodo `run()` 15
3.4 Preprocessing dei Dati 15
3.4.1 Split Train/Validation 15
3.4.2 Normalizzazione 15
3.4.3 One-Hot Encoding 16

4 Concetti Trasversali Importanti **17**
4.1 Overfitting 17
4.2 Classificazione vs Regressione 17
4.3 Generalizzabilità dei Risultati 18

5 Esperimenti e Risultati **19**
5.1 Test 1 — Mutation Rate 19
5.2 Test 2 — Lambda (Penalità di Complessità) 20
5.3 Test 3 — Numero di Epoche 21

5.4 Test 4 — Learning Rate 22

6 Discussione e Conclusioni **24**

6.1 Analisi Complessiva 24

6.2 Limiti del Progetto 24

6.3 Sviluppi Futuri 24

A Glossario dei Termini Chiave **26**

1 Introduzione

Il progetto affronta il problema della **Neural Architecture Search (NAS)**, ovvero la ricerca automatica della migliore architettura di una rete neurale per un dato problema. Tradizionalmente, la progettazione di una rete neurale richiede che il ricercatore scelga manualmente il numero di layer, il numero di neuroni per layer e le funzioni di attivazione. Questa scelta è spesso guidata dall'esperienza e da tentativi ed errori, rendendo il processo lungo e dipendente dall'expertise del progettista.

L'obiettivo di questo progetto è automatizzare questa scelta utilizzando un **Algoritmo Genetico**, che esplora lo spazio delle possibili architetture e converge verso quelle che massimizzano l'accuratezza sul dataset Iris, tenendo conto anche della complessità computazionale della rete tramite una funzione di penalità.

1.1 Originalità e Rilevanza Pratica

Il progetto non è originale nel senso accademico del termine — la NAS esiste dagli anni '90. Tuttavia il suo contributo è dimostrare la comprensione profonda di due paradigmi fondamentali dell'AI e la capacità di farli interagire. Il filone di ricerca esplorato è oggi uno dei più attivi nel campo: modelli come *EfficientNet* di Google nascono da processi automatici di ricerca dell'architettura, e la NAS è ampiamente usata in contesti industriali dove si deve deployare una rete su hardware con risorse limitate.

Possibile domanda d'esame

"Il tuo progetto è originale?"

No, nel senso accademico stretto. La NAS tramite GA è una tecnica nota. Ciò che è originale è la combinazione specifica: una rete neurale scritta da zero con backpropagation implementata manualmente, integrata con un GA che ne ottimizza l'architettura, con penalità di complessità e valutazione multipla (K=3) per ridurre la varianza. Tutti i componenti sono costruiti senza librerie di machine learning preconfezionate.

2 Background Teorico

2.1 Reti Neurali Artificiali

Una rete neurale artificiale è un modello computazionale ispirato al funzionamento del cervello biologico. È composta da unità elementari chiamate **neuroni**, organizzati in layer successivi. Ogni neurone riceve input dal layer precedente, li combina linearmente tramite pesi e bias, e applica una funzione di attivazione non lineare per produrre il proprio output.

2.1.1 Struttura di un Neurone

Per un singolo neurone l'output è calcolato come:

$$\text{output} = f(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{1}$$

dove $\mathbf{W}$ è la matrice dei pesi, $\mathbf{x}$ il vettore di input, $\mathbf{b}$ il vettore dei bias e f la funzione di attivazione.

La struttura di una rete feed-forward è composta da tre tipologie di layer:

- **Layer di input** — riceve le feature del dataset senza applicare nessuna trasformazione.
- **Hidden layer** — trasforma la rappresentazione dei dati applicando combinazioni lineari seguite da funzioni di attivazione non lineari.
- **Layer di output** — produce la predizione finale con una funzione di attivazione appropriata al tipo di problema.

2.1.2 Funzioni di Attivazione

Le funzioni di attivazione introducono la non linearità nella rete. Senza di esse, layer multipli sarebbero equivalenti a un singolo layer lineare — indipendentemente dalla profondità, la rete potrebbe solo rappresentare trasformazioni lineari dell'input, incapace di apprendere pattern complessi.

ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x), \qquad f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases} \tag{2}$$

Ideale per gli hidden layer: computazionalmente efficiente e non soffre del problema del gradiente che svanisce per valori positivi. Ha però il problema dei *neuroni morti* —

se l'input è sempre negativo il neurone smette di attivarsi e il suo gradiente è sempre zero.

Leaky ReLU

$$f(x) = \max(0.01x,\ x), \qquad f'(x) = \begin{cases} 1 & x > 0 \\ 0.01 & x \leq 0 \end{cases} \tag{3}$$

Variante di ReLU che risolve il problema dei neuroni morti permettendo un piccolo gradiente anche per valori negativi.

Sigmoid

$$f(x) = \frac{1}{1 + e^{-x}}, \qquad f'(x) = f(x)\big(1 - f(x)\big) \tag{4}$$

Comprime l'output nell'intervallo $(0, 1)$. Soffre del problema del gradiente che svanisce per valori molto grandi o molto piccoli (*vanishing gradient*).

Softmax

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \tag{5}$$

Trasforma un vettore di valori arbitrari in una distribuzione di probabilità (tutti i valori tra 0 e 1, somma = 1). È la funzione ideale per il layer di output in problemi di **classificazione multi-classe** come Iris.

Perché Softmax solo sull'output layer?

La Softmax è definita su un vettore e il suo output dipende da *tutti* i valori del vettore contemporaneamente. Non ha senso applicarla a un singolo neurone isolato in un hidden layer, perché l'operazione $e^{x_i} / \sum_j e^{x_j}$ su un singolo numero darebbe sempre 1. Negli hidden layer si usano funzioni elemento-per-elemento come ReLU e Sigmoid.

Possibile domanda d'esame

"Perché usi l'approssimazione diagonale della derivata della Softmax?"
La derivata completa della Softmax è una matrice — il Jacobiano — perché ogni output dipende da tutti gli input. L'elemento (i, j) del Jacobiano è:

$$\frac{\partial f_i}{\partial x_j} = \begin{cases} f_i(1 - f_i) & i = j \\ -f_i f_j & i \neq j \end{cases}$$

Nell'implementazione si usa solo la diagonale $f_i(1-f_i)$, che coincide con la formula della Sigmoid. Questa approssimazione evita di dover modificare la struttura della backpropagation (che si aspetta vettori, non matrici) ed è sufficiente perché su Iris le classi sono ben separabili e la rete converge comunque correttamente.

2.1.3 Backpropagation

La backpropagation è l'algoritmo standard per addestrare reti neurali tramite discesa del gradiente. Funziona in due fasi.

Forward pass — l'input viene propagato attraverso tutti i layer fino all'output. I valori intermedi (pre-attivazione $\mathbf{z}$ e post-attivazione $\mathbf{a}$) vengono salvati perché necessari nella fase successiva.

Backward pass — l'errore viene propagato all'indietro calcolando il gradiente della funzione di loss rispetto a ogni peso tramite la **regola della catena**:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \cdot \left(\mathbf{a}^{(l-1)}\right)^T \tag{6}$$

dove $\boldsymbol{\delta}^{(l)}$ è il vettore dei delta del layer l.

La funzione di loss usata è il **Mean Squared Error (MSE)**:

$$\mathcal{L} = \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2 , \qquad \frac{\partial \mathcal{L}}{\partial \hat{y}_i} = \hat{y}_i - y_i \tag{7}$$

I pesi vengono aggiornati sottraendo il gradiente moltiplicato per il learning rate η:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{W}} \tag{8}$$

Perché salvare i valori intermedi?

L'implementazione mantiene `valori_intermedi` e `valori_input` come attributi della rete perché il calcolo dei delta nella fase backward richiede i valori pre-attivazione $\mathbf{z}^{(l)}$ (per applicare f') e le attivazioni $\mathbf{a}^{(l-1)}$ (per calcolare il gradiente dei pesi). Senza salvarli durante il forward pass bisognerebbe ricalcolarli, raddoppiando il costo computazionale.

2.2 Algoritmi Genetici

Gli algoritmi genetici sono metodi di ottimizzazione ispirati alla teoria dell'evoluzione naturale di Darwin. Operano su una **popolazione** di soluzioni candidate e le fanno evolvere attraverso generazioni successive applicando operatori biologicamente ispirati.

2.2.1 Concetti Fondamentali

- **Cromosoma** — rappresentazione di una soluzione candidata. Nel progetto è una lista di tuple che descrive l'architettura della rete.
- **Gene** — elemento atomico del cromosoma. Nel progetto ogni gene è una tupla `(n_neuroni, funzione_attivazione)`.
- **Fitness** — misura della qualità di ogni individuo. Nel progetto è l'accuratezza sul validation set penalizzata per la complessità.
- **Generazione** — un'iterazione completa del ciclo evolutivo.
- **Popolazione** — l'insieme di tutti gli individui in una generazione.

2.2.2 Il Ciclo Evolutivo

1. Generazione casuale della popolazione iniziale
2. Valutazione della fitness di ogni individuo
3. Selezione dei genitori (torneo)
4. Generazione dei figli tramite crossover
5. Mutazione dei figli
6. Sostituzione della vecchia popolazione con la nuova
7. Ripeti dal passo 2 per N generazioni

2.2.3 Selezione Tramite Torneo

La **selezione a torneo** bilancia due esigenze opposte che un buon GA deve avere contemporaneamente:

- **Sfruttamento** (*exploitation*) — i migliori individui devono avere più probabilità di riprodursi.
- **Esplorazione** (*exploration*) — anche individui mediocri devono avere qualche possibilità, per mantenere diversità genetica e ridurre il rischio di convergenza prematura verso ottimi locali.

Possibile domanda d'esame

"Perché usi la selezione a torneo invece di selezionare sempre i migliori?"
Se si selezionassero sempre i migliori, dopo poche generazioni tutta la popolazione diventerebbe molto simile. Il GA smette di esplorare e converge prematuramente su un ottimo locale. Con il torneo, estraiamo $k = 3$ individui a caso e scegliamo il migliore tra loro: un individuo forte ha alta probabilità di vincere, ma uno mediocre ha ancora chance se gli capita un torneo con avversari più deboli. La dimensione k controlla il bilanciamento: k grande $\Rightarrow$ più pressione selettiva, k piccolo $\Rightarrow$ più esplorazione.

2.3 Neural Architecture Search

La NAS è il campo di ricerca che studia come automatizzare la progettazione di architetture di reti neurali. Questo progetto ne implementa una versione semplificata: un GA che esplora lo spazio delle architetture usando come funzione di fitness l'accuratezza ottenuta dopo training con backpropagation.

Il concetto chiave è la **separazione tra due livelli di ottimizzazione**:

- Il **GA** ottimizza l'architettura — parametri *discreti* come numero di layer, neuroni e funzioni di attivazione. Non esiste un gradiente per questi parametri, quindi la discesa del gradiente non è applicabile.
- La **backpropagation** ottimizza i pesi all'interno di ogni architettura — parametri *continui*. Opera su scale temporali diverse e con un meccanismo completamente diverso.

Possibile domanda d'esame

"Il tuo progetto usa il Reinforcement Learning?"

No. Il RL è un paradigma in cui un agente impara agendo in un ambiente e ricevendo reward. Il progetto usa un GA (ottimizzazione evolutiva) combinato con backpropagation (ottimizzazione per gradiente). I modelli generativi come le GAN o gli LLM sono anch'essi un campo separato. Il punto di contatto con il RL esiste solo in alcune applicazioni avanzate (es. RLHF per LLM) che non riguardano questo progetto.

3 Implementazione

3.1 Struttura del Progetto

- `nn_layer.py` — classe `layer`, funzioni di attivazione e derivate
- `nn_engine.py` — classe `neural_network` con feedforward e backpropagation
- `genetic_algorithm.py` — classe `GeneticAlgorithm` con tutti gli operatori
- `main.py` — preprocessing, baseline, esecuzione GA e grafici
- `test_*.py` — script per i quattro esperimenti di sensibilità

3.2 Rete Neurale (`nn_engine.py`)

3.2.1 Modifica al Costruttore

La modifica principale rispetto alla versione originale riguarda il costruttore della classe `neural_network`. Il parametro `size_hidden` — che imponeva lo stesso numero di neuroni a tutti gli hidden layer — è stato sostituito con `hidden_config`, una lista di tuple:

```python
def __init__(self, hidden_config, size_input, size_output,
             learning_rate, output_function):
    self.size = len(hidden_config) + 1
    self.hiddens_layers = []
    prev = size_input
    for n_neuroni, f in hidden_config:
        self.hiddens_layers.append(layer(n_neuroni, prev, f
           ))
        prev = n_neuroni
    self.output_layer = layer(size_output, prev,
       output_function)
    self.valori_intermedi = []
    self.valori_input = []
```

Listing 1: Nuovo costruttore della rete neurale

La variabile `prev` tiene traccia del numero di neuroni del layer precedente, che determina le colonne della matrice dei pesi del layer successivo. La matrice pesi di ogni layer ha forma (n_neuroni_corrente × n_neuroni_precedente).

3.2.2 Funzioni di Attivazione per Layer

Le funzioni sono organizzate in due dizionari distinti:

```
# Solo per hidden layer
hidden_functions = [relu, leaky_relu, sigmoid]

# Solo per output layer
functions_output = {linear: d_linear,
                    sigmoid: d_sigmoid,
                    softmax: d_softmax}
```

Listing 2: Separazione tra funzioni per hidden e output layer

Questa separazione riflette una scelta progettuale precisa: ReLU e Leaky ReLU non sono adatte all'output layer perché non producono distribuzioni di probabilità. La Softmax non è adatta agli hidden layer perché opera sull'intero vettore e non ha senso neurone per neurone.

3.3 Algoritmo Genetico (`genetic_algorithm.py`)

3.3.1 Rappresentazione del Cromosoma

Un individuo è una lista Python di tuple:

```
[(23, relu), (16, sigmoid)]          # 2 hidden layer
[(32, relu), (16, relu), (8, relu)]  # 3 hidden layer
[(8, leaky_relu)]                    # 1 solo hidden layer
```

Listing 3: Esempi di cromosomi

Il GA può far variare il numero di layer tra individui diversi. La funzione di output è fissa a `softmax` per tutti gli individui.

3.3.2 Funzione di Fitness con Penalità di Complessità

```
def _fitness(self, individuo):
    sum_accuracy = 0
    for _ in range(self.K):          # K=3 run indipendenti
        n = neural_network(individuo, 4, 3,
                           self.learning_rate, softmax)
        for _ in range(self.epochs): # training con SGD
            x = random.randint(0, len(self.Y_train) - 1)
            n.feedback(self.X_train[x], self.Y_train[x])
        correct = 0
        for i in range(len(self.Y_val)):
            guess = n.feedforward(self.X_val[i])
```

```
            if np.argmax(guess) == np.argmax(self.Y_val[i]):
                correct += 1
        sum_accuracy += correct / len(self.X_val)
    accuracy = sum_accuracy / self.K
    penalita = self._complessita(individuo)
    fitness  = accuracy - self.lambda_ * penalita
    return (fitness, accuracy)      # restituite entrambe
```

Listing 4: Funzione di fitness

La fitness è calcolata come:

$$\text{fitness} = \text{accuracy} - \lambda \cdot N_{\text{params}} \tag{9}$$

dove N_{params} è il numero totale di pesi e bias della rete. Fitness e accuracy vengono restituite separatamente: il GA usa la fitness per la selezione, ma nei grafici si mostra sempre l'accuracy pura per mantenere la leggibilità. Per definizione:

$$\text{fitness} \leq \text{accuracy} \quad \forall \lambda > 0 \tag{10}$$

Perché K

I pesi della rete vengono inizializzati casualmente ad ogni valutazione. Con una sola run, un'architettura buona potrebbe ricevere una fitness bassa per pura sfortuna nell'inizializzazione. Con K=3 run indipendenti e media delle accuratezze, la varianza si riduce significativamente senza triplicare completamente il tempo di esecuzione.

Possibile domanda d'esame

"I pesi si ereditano dai genitori?"

No. Il cromosoma rappresenta solo l'architettura (struttura discreta). I pesi non fanno parte del cromosoma e vengono inizializzati casualmente ogni volta che si costruisce una rete durante la valutazione della fitness. In biologia: i geni trasmettono l'istruzione per costruire l'organismo, non l'organismo già costruito. L'architettura è il gene, i pesi sono le caratteristiche fisiche sviluppate durante il training.

3.3.3 Crossover

```
def _crossover(self, genitore1, genitore2):
    t1 = random.randint(0, len(genitore1))
    t2 = random.randint(0, len(genitore2))
```

```
    figlio = genitore1[:t1] + genitore2[t2:]
    if len(figlio) == 0:            # caso degenere
        pool = genitore1 + genitore2
        figlio.append(random.choice(pool))
    return figlio
```

Listing 5: Crossover a doppio punto di taglio

I punti di taglio sono indipendenti per gestire genitori con lunghezze diverse. Il limite superiore è `len(genitore)` incluso, così lo slicing può restituire l'intera lista (`lista[:n]` = `lista` se $n = \text{len}$).

3.3.4 Mutazione

```
def _mutazione(self, individuo):
    individuo = [layer_ for layer_ in individuo]  # copia!
    for i in range(len(individuo)):
        if random.random() < self.mutation_rate:
            mut = random.choices([0, 1, 2], weights
               =[1,1,1])[0]
            if mut == 0:   # cambia n. neuroni
                individuo[i] = (random.randint(4,32),
                                individuo[i][1])
            elif mut == 1: # cambia funzione
                individuo[i] = (individuo[i][0],
                                random.choice(self.
                                   hidden_functions))
            else:          # mutazione strutturale
                if random.randint(0,1)==0 and len(individuo
                   )>1:
                    del individuo[i]
                    break  # indici non piu' validi dopo
                       del
                else:
                    pos = random.randint(0, len(individuo))
                    individuo.insert(pos,
                        (random.randint(4,32),
                         random.choice(self.
                            hidden_functions)))
    return individuo
```

Listing 6: Mutazione con tre tipi di modifica

Perché lavorare su una copia in __mutazione?

In Python le liste sono passate **per riferimento**, non per valore. Se si modifica direttamente `individuo` si altera la lista originale nella popolazione, potendo corrompere i genitori anche dopo il crossover. La lista di comprensione `[layer_ for layer_ in individuo]` crea una copia superficiale: le tuple interne sono immutabili quindi non servono copie profonde.

3.3.5 Elitismo nel metodo `run()`

Il miglior individuo trovato fino alla generazione corrente viene copiato direttamente nella nuova popolazione prima di generare i figli. Questo garantisce che la qualità della soluzione migliore non possa mai peggiorare tra generazioni, anche in presenza di mutazioni sfortunate.

3.4 Preprocessing dei Dati

Il dataset Iris contiene 150 campioni, 4 feature e 3 classi.

3.4.1 Split Train/Validation

```
x_train, x_val, y_train, y_val = train_test_split(
    x, y, test_size=0.2, random_state=42)
```

Listing 7: Split 80/20 con seed fisso

`random_state=42` garantisce la riproducibilità: lo stesso split ad ogni esecuzione permette confronti coerenti tra run diverse.

3.4.2 Normalizzazione

```
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)  # calcola media e
    std
x_val   = scaler.transform(x_val)        # applica le
    stesse
```

Listing 8: StandardScaler applicato correttamente

Possibile domanda d'esame

"Perché non calcoli media e deviazione standard anche sul validation set?"
Questo introdurrebbe **data leakage**: stai usando informazioni sui dati futuri per trasformare i dati presenti. In produzione il validation set arriva dopo il training e non è disponibile in anticipo. Se usassi le sue statistiche la valutazione delle prestazioni sarebbe falsata — tipicamente ottimistica. La regola è sempre: `fit` solo su training set, `transform` su tutto il resto con le stesse statistiche.

3.4.3 One-Hot Encoding

Le etichette intere (0, 1, 2) vengono convertite in vettori binari:

$$0 \to [1, 0, 0], \quad 1 \to [0, 1, 0], \quad 2 \to [0, 0, 1]$$

Possibile domanda d'esame

"Perché non usi semplicemente 0, 1, 2 come etichette?"
Questo introdurrebbe un **ordine artificiale** tra le classi: la rete potrebbe imparare che Virginica (2) è "più grande" di Setosa (0), il che non ha senso biologico. Con il one-hot encoding le tre classi sono trattate in modo simmetrico, ognuna con il proprio neurone di output dedicato.

4 Concetti Trasversali Importanti

4.1 Overfitting

Un modello in overfitting ha **imparato a memoria** i dati di training invece di imparare le regole generali sottostanti. Funziona perfettamente sui dati visti durante l'addestramento, ma fallisce su dati nuovi.

Il segnale classico è una discrepanza tra le due accuracy:

Accuracy su training set	99%
Accuracy su validation set	65%

Il modello ha troppa **capacità** rispetto alla complessità del problema — troppi parametri rispetto ai dati disponibili.

Collegamento con il progetto

Con $\lambda = 0$ il GA trova una rete con 7 layer e 5447 parametri su soli 120 campioni di training: condizioni ideali per l'overfitting. Il fatto che l'accuracy sul validation sia comunque 98.89% suggerisce che Iris è così semplice da non causare overfitting nemmeno con reti enormi. Ma su dataset più complessi la stessa architettura probabilmente fallirebbe. La penalità λ serve proprio a prevenire questo problema favorendo architetture più semplici.

Le tecniche principali per combattere l'overfitting sono:

- **Regolarizzazione** (implementata) — penalizzare la complessità tramite λ.
- **Dropout** (non implementato) — spegnere casualmente neuroni durante il training.
- **Early stopping** (non implementato) — interrompere il training quando l'accuracy sul validation smette di migliorare.

4.2 Classificazione vs Regressione

- **Classificazione** — l'output è una categoria discreta. Iris è classificazione: "a quale delle 3 specie appartiene questo fiore?". Si usa Softmax sull'output e si misura l'accuratezza.
- **Regressione** — l'output è un valore continuo. Es: "quanto costerà questa casa?". Si usa Linear sull'output e si misura MSE/RMSE.

4.3 Generalizzabilità dei Risultati

Possibile domanda d'esame

"L'architettura trovata su Iris funzionerebbe su altri dataset?"

No. Il GA trova la migliore architettura *per quel dataset specifico*. L'architettura ottimale dipende dalla complessità del problema: numero di feature, separabilità delle classi, quantità di dati. Il *metodo* (GA + NAS) è generalizzabile, ma il *risultato* (l'architettura trovata) è specifico per Iris. Su dataset più complessi servirebbero più generazioni, popolazione più grande e architetture più profonde.

5 Esperimenti e Risultati

Sono stati condotti quattro esperimenti variando un singolo parametro alla volta (analisi di sensibilità). La configurazione base è:

Parametro	Valore
`population_size`	20
`generations`	30
`mutation_rate`	0.2
`learning_rate`	0.01
`lambda_`	0.00005
`epochs`	150
`tournament_size`	3
`K`	3
`EPOCHS_BASELINE`	450

Le epoche della baseline sono state fissate a $K \times \text{epochs} = 3 \times 150 = 450$: stesso numero di epoche usato per valutare un singolo individuo, confronto equo.

5.1 Test 1 — Mutation Rate

mutation_rate	best_accuracy	best_fitness	n_layer	n_params
0.05	94.44%	83.27	3	2235
0.20	92.22%	85.73	3	1298
0.40	91.11%	84.25	3	1372
Baseline	70.00%	—	2	—

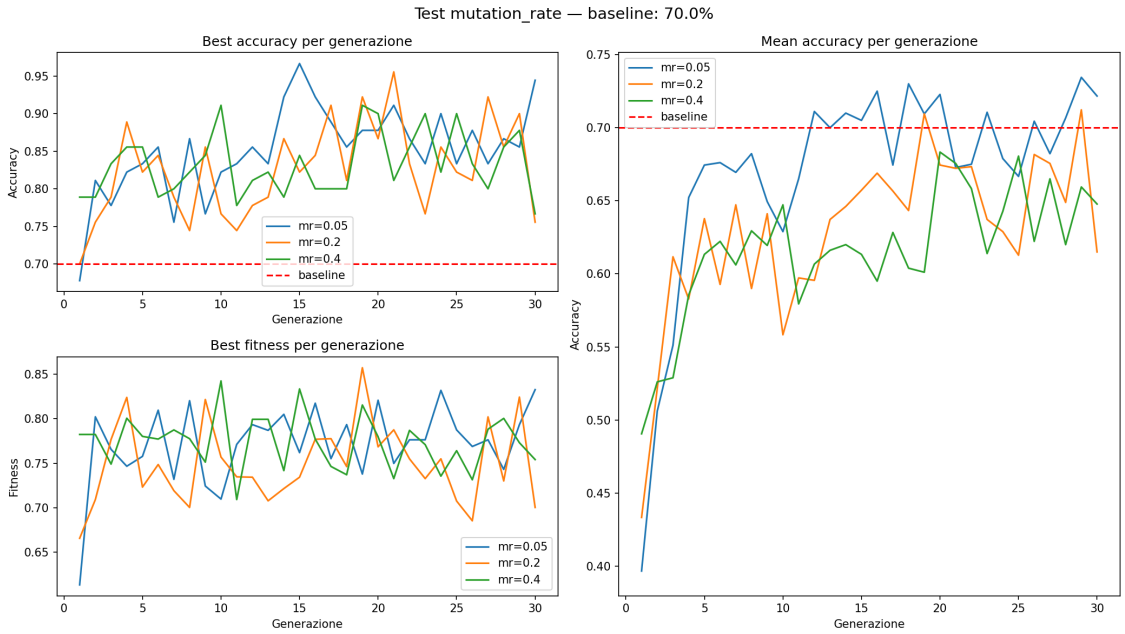

Figura 1: Evoluzione della fitness e accuracy al variare del mutation rate.

Osservazioni principali:

Tutte le run battono la baseline con miglioramenti tra +21% e +24%. Questo valida l'approccio indipendentemente dal mutation rate scelto.

Mutation rate basso (0.05) trova la migliore accuracy (94.44%) ma genera architetture più complesse (2235 parametri), risultando nella fitness più bassa (83.27) per via della penalità. Con poca mutazione il GA si affida al solo crossover, che tende a preservare reti grandi.

Mutation rate medio (0.20) ottiene la fitness più alta (85.73) con architettura più snella (1298 parametri): miglior compromesso tra accuratezza e complessità.

Mutation rate alto (0.40) introduce troppo rumore, rendendo difficile accumulare miglioramenti. La curva della mean accuracy è la più bassa e rumorosa lungo tutta la run.

Valore ottimale: ***mutation_rate = 0.2***

5.2 Test 2 — Lambda (Penalità di Complessità)

lambda	best_accuracy	best_fitness	n_layer	n_params
0.0	98.89%	98.89	7	5447
0.00005	88.89%	87.83	1	211
0.0002	84.44%	80.06	1	219
Baseline	60.00%	—	2	—

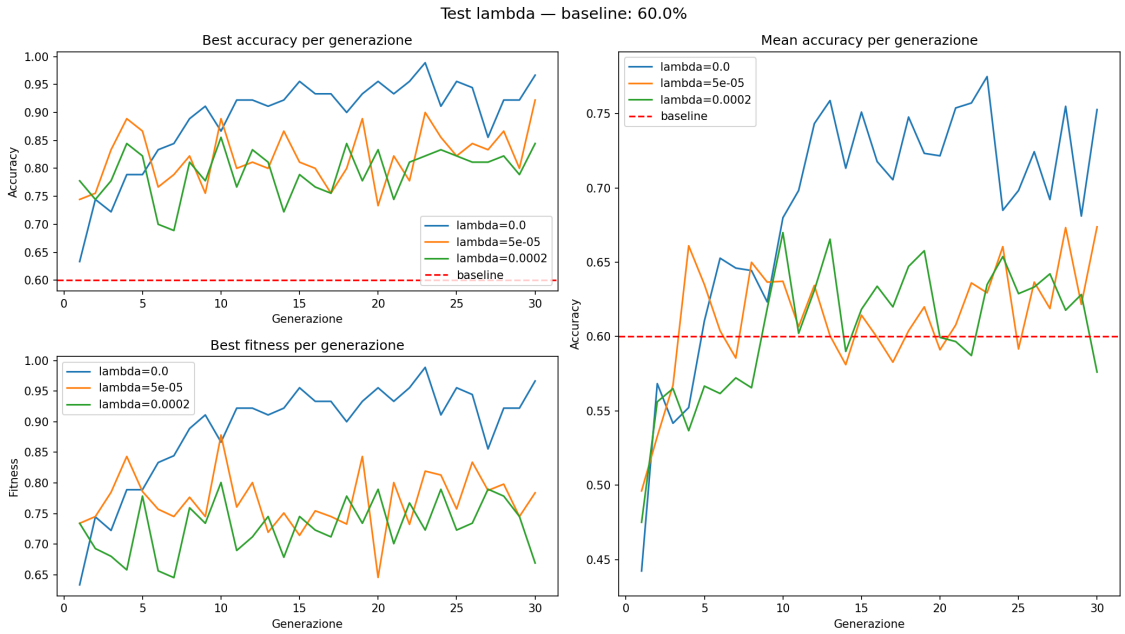

Figura 2: Evoluzione della fitness e accuracy al variare di lambda.

Osservazioni principali:

Con $\lambda = 0$ fitness e accuracy coincidono (nessuna penalità) e il GA trova 7 layer con 5447 parametri: massimizza solo l'accuracy senza nessun costo per la complessità. Condizioni ideali per overfitting su dataset più complessi.

Con $\lambda = 0.00005$ la complessità si riduce del 96%: da 5447 a 211 parametri. Anche una penalità piccola è sufficiente perché su 5447 parametri vale $0.00005 \times 5447 = 0.27$ (27 punti di accuracy sottratti).

Con $\lambda = 0.0002$ il GA trova architetture troppo semplici sacrificando troppa accuracy. Lambda è troppo aggressivo.

Valore ottimale: ***lambda = 0.00005***

5.3 Test 3 — Numero di Epoche

epochs	best_accuracy	best_fitness	n_layer	n_params
50	75.56%	74.30	1	251
150	90.00%	88.75	1	251
300	92.22%	91.13	1	219
Baseline	70.00%	—	2	—

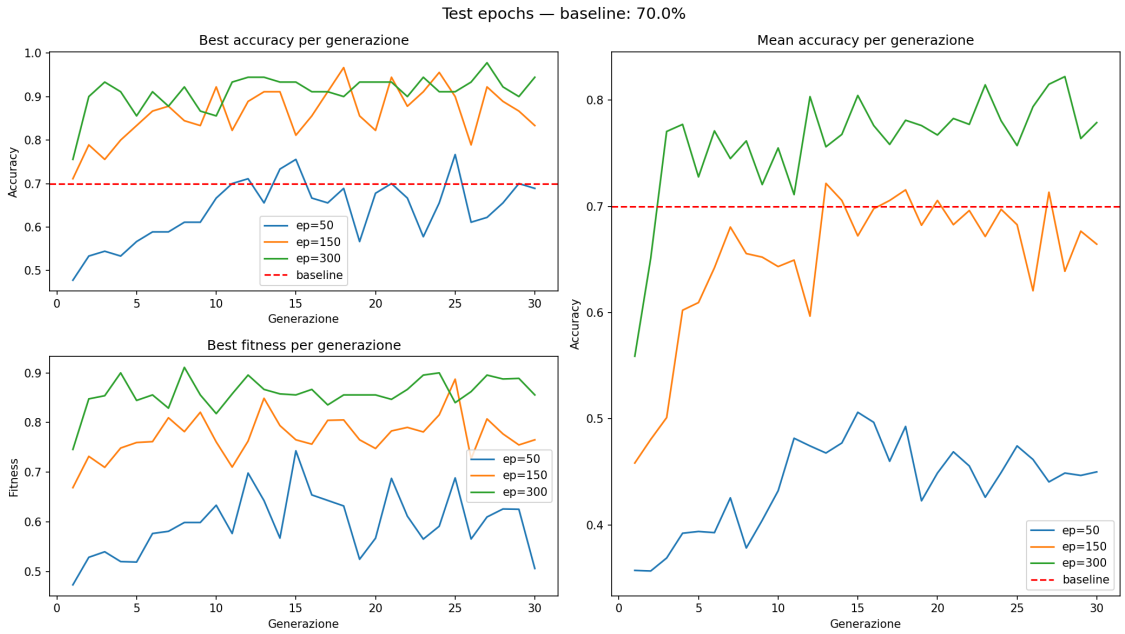

Figura 3: Evoluzione della fitness e accuracy al variare del numero di epoche.

Osservazioni principali:

Con 50 epoche la backpropagation non converge: i pesi restano quasi casuali e la fitness non è informativa. Il GA non riesce a distinguere architetture buone da cattive e non evolve efficacemente.

Da 150 a 300 epoche il miglioramento è solo +2.22% a fronte di un raddoppio del tempo totale di esecuzione ($20 \times 3 \times 300 \times 30 = 54.000$ training). Caso classico di **rendimenti decrescenti**.

Valore ottimale: ***epochs = 150***

5.4 Test 4 — Learning Rate

learning_rate	best_accuracy	best_fitness	n_layer	n_params
0.001	60.00%	59.35	1	131
0.010	90.00%	85.36	2	928
0.050	96.67%	95.65	1	203
Baseline	70.00%	—	2	—

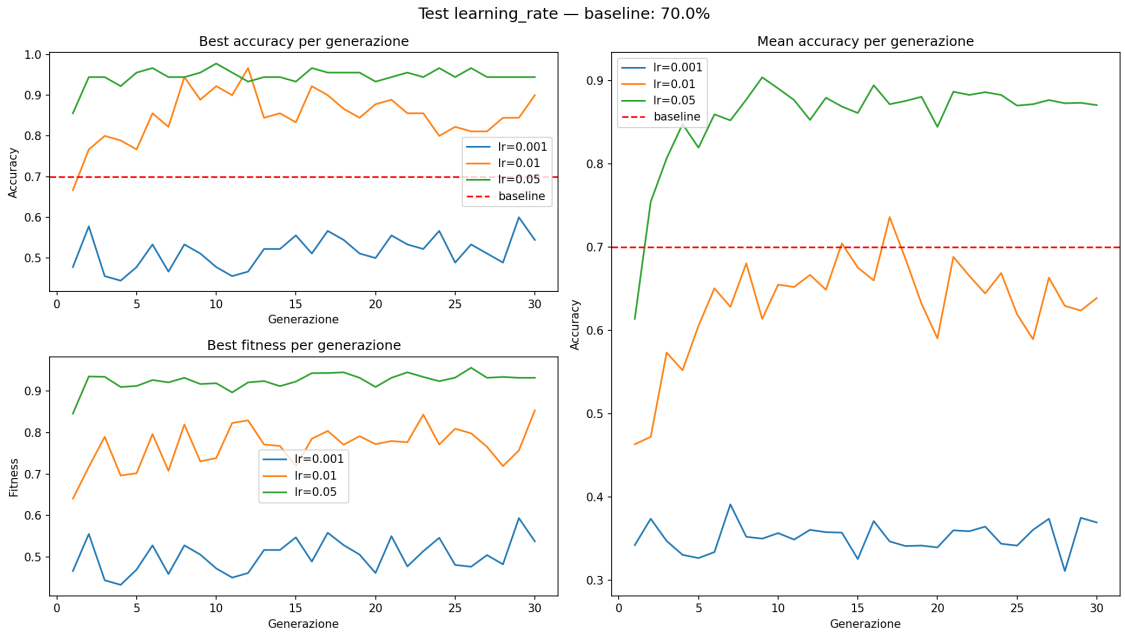

Figura 4: Evoluzione della fitness e accuracy al variare del learning rate.

Osservazioni principali:

Il learning rate è il parametro più critico dell'intero sistema: la differenza tra caso peggiore e migliore è di 36.67 punti percentuali di accuracy, il range più ampio tra tutti i test.

Con lr=0.001 il GA non supera la baseline: la rete non converge in 150 epoche, la fitness è rumore casuale e il GA non può evolvere.

Con lr=0.05 si ottiene il miglior risultato in assoluto: 96.67% con 1 solo layer e 203 parametri.

Rischio di divergenza: lr troppo alto può causare oscillazioni su problemi più complessi — i passi di aggiornamento sono così grandi che i pesi "saltano" oltre il minimo della funzione di errore.

Parallelismo con le epoche: learning rate e numero di epoche sono parametri complementari. Entrambi controllano la convergenza:

$$\text{convergenza} = f(\text{lr},\ \text{epochs})$$

Un lr basso può essere compensato da più epoche, ma al costo del tempo.

Valore ottimale: ***learning_rate = 0.05***

6 Discussione e Conclusioni

6.1 Analisi Complessiva

Il GA supera la baseline in quasi tutte le configurazioni testate. Il risultato migliore in assoluto — 96.67% di accuracy con lr=0.05 — è significativamente superiore alla baseline del 70%. Il GA tende a trovare architetture con 1 solo hidden layer quando la penalità è attiva, confermando che Iris è linearmente quasi separabile e non richiede architetture profonde.

La tabella seguente riassume i valori ottimali identificati:

Parametro	Valore ottimale	Motivazione
mutation_rate	0.20	Miglior compromesso esplorazione/sfruttamento
lambda_	0.00005	Riduce complessità senza sacrificare accuracy
epochs	150	Convergenza affidabile, tempo accettabile
learning_rate	0.05	Convergenza rapida in 150 epoche

6.2 Limiti del Progetto

- **Generalizzabilità**: le architetture ottimali trovate sono specifiche per Iris. Su dataset più complessi il GA richiederebbe più generazioni e popolazione più grande.
- **Variabilità**: i risultati variano tra run diverse a causa della casualità. Per risultati affidabili servirebbero multiple run con media e deviazione standard.
- **Ottimizzatore**: la backpropagation usa SGD puro senza ottimizzatori moderni (Adam, RMSprop) che garantirebbero convergenza più rapida e stabile.
- **Derivata Softmax**: l'approssimazione diagonale del Jacobiano potrebbe essere limitante su problemi più complessi.

6.3 Sviluppi Futuri

- Validazione su dataset più complessi (es. MNIST, Wine Quality)
- Implementazione del Jacobiano completo per la Softmax
- Aggiunta di Dropout per ridurre il rischio di overfitting
- Implementazione di Early Stopping nella valutazione della fitness
- Implementazione di Adam come ottimizzatore alternativo a SGD

- Esposizione di K come parametro configurabile del costruttore

A Glossario dei Termini Chiave

Accuratezza (Accuracy) Percentuale di predizioni corrette sul validation set. Valore tra 0 e 1.

Backpropagation Algoritmo per calcolare il gradiente della funzione di loss rispetto ai pesi della rete, propagando l'errore all'indietro layer per layer tramite la regola della catena.

Baseline Rete con architettura fissa $[(8, \text{relu}), (8, \text{relu})]$ addestrata con backpropagation classica, usata come punto di riferimento per valutare i risultati del GA.

Classificazione Problema in cui l'output è una categoria discreta. Contrapposto alla regressione (output continuo).

Cromosoma Rappresentazione di una soluzione candidata nel GA: lista di tuple `(n_neuroni, funzione_attivazione)`.

Crossover Operatore genetico che combina due genitori per generare un figlio scambiando parti dei cromosomi.

Data leakage Uso di informazioni del validation set durante il preprocessing o il training, che porta a stime ottimistiche delle prestazioni.

Elitismo Tecnica che copia il miglior individuo nella nuova popolazione, garantendo che la qualità non peggiori tra generazioni.

Fitness Misura della qualità di un individuo nel GA: $\text{fitness} = \text{accuracy} - \lambda \cdot N_{\text{params}}$.

Gradient descent Algoritmo di ottimizzazione che aggiorna i parametri nella direzione opposta al gradiente della funzione di loss.

Learning rate (η) Iperparametro che controlla la dimensione del passo di aggiornamento dei pesi durante la backpropagation.

Mutazione Operatore genetico che introduce variazioni casuali nel cromosoma per mantenere diversità nella popolazione.

NAS (Neural Architecture Search) Campo di ricerca che studia l'automazione della progettazione di architetture di reti neurali.

One-hot encoding Rappresentazione delle etichette come vettori binari con un solo 1 nella posizione corrispondente alla classe.

Overfitting Fenomeno per cui il modello memorizza i dati di training e non generalizza su dati nuovi.

Penalità di complessità Termine $\lambda \cdot N_{\text{params}}$ sottratto dalla fitness per favorire architetture semplici.

Pressione selettiva Intensità con cui gli individui migliori vengono preferiti nella riproduzione.

Rendimenti decrescenti Fenomeno per cui ogni unità aggiuntiva di una risorsa contribuisce sempre meno al miglioramento.

Selezione a torneo Metodo che estrae K individui casuali e sceglie il migliore tra loro come genitore.

Softmax Funzione di attivazione che normalizza un vettore in una distribuzione di probabilità. Usata solo sull'output layer per classificazione multi-classe.

StandardScaler Trasformazione che porta ogni feature a media 0 e deviazione standard 1.

Validation set Porzione del dataset non usata durante il training, per valutare le prestazioni su dati mai visti.

Vanishing gradient Problema per cui i gradienti diventano estremamente piccoli nelle prime fasi del training, rallentando o impedendo l'apprendimento (tipico di Sigmoid su reti profonde).